



Query Templating & CWE Knowledge Base (1)

Estensione avanzata dell'agente: invece di far scrivere CodeQL al modello, gli si fanno riempire **template name-based** validati, guidato da una **knowledge base CWE a due layer**. Pagina complementare alla [Setup Guide](#).

1. L'idea centrale

Scrivere query CodeQL è difficile per modelli piccoli. Soluzione: l'agente **non scrive QL**, riempie dei buchi name-based (liste di nomi di funzioni/metodi) in template già validi. Le modifiche name-based, se validate con regex, **non possono rompere la sintassi**.

Flusso operativo:

```
list_cwes -> cwe_knowledge(cwe)           # consapevolezza: co
s'è + nomi candidati + tool giusto
|
read_file_snippet(...)                   # vede i nomi reali nel
codice
|
taint? -> run_taint_query(cwe, sink_names...) #
flusso source->sink
crypto? -> run_api_misuse_query(cwe, weak_call_names...)
# uso API debole
|
CWE inciso nel SARIF -> report ancorato all'evidenza determin
istica
```

2. Famiglie di vulnerabilità supportate

Famiglia	Approccio	Template / Tool
Taint / injection	source → sink data flow	<code>taint_namebased.ql.tpl</code> • <code>run_taint_query</code>
Crittografia	rilevamento API deboli per nome (point detection)	<code>api_misuse_namebased.ql.tpl</code> • <code>run_api_misuse_query</code>
Use-after-free / memory	✗ non supportato	problemi C/C++, non esprimibili per nome; servono query memory dedicate su DB C/C++

CWE coperti (16)

- **Taint (12):** CWE-89 (SQLi), CWE-78 (OS command), CWE-22 (path traversal), CWE-79 (XSS), CWE-502 (deserialization), CWE-90 (LDAP), CWE-94 (code/eval), CWE-611 (XXE), CWE-643 (XPath), CWE-601 (open redirect), CWE-918 (SSRF), CWE-117 (log injection).
- **Crypto (4):** CWE-328 (weak hash), CWE-327 (broken cipher), CWE-330 (weak random), CWE-916 (password hash).

3. Template name-based

3.1 Taint (`query_templates/taint_namebased.ql.tpl`)

Template con 3 segnaposto `{{SINK_NAMES}}` / `{{SOURCE_NAMES}}` / `{{SANITIZER_NAMES}}` + placeholder CWE. Punti chiave:

- Helper `callNameIn` matcha sia `obj.NAME(...)` (AttrRead) sia `NAME(...)` (bare call) → copre wrapper custom e funzioni dirette (`eval` , `system`).
- **Sanitizer = `isBarrier` , NON `isSanitizer`** (nel nuovo dataflow API `isSanitizer` viene ignorato silenziosamente in `DataFlow::ConfigSig` + `TaintTracking::Global`).
- Sanitizer **opzionale**: lista vuota → sentinella `__never_matches__` , così `in [...]` resta valido e non matcha nulla.

3.2 Crypto (`query_templates/api_misuse_namebased.ql.tpl`)

@kind problem (point detection, niente dataflow). Flagga:

- chiamate a API deboli per nome (`{{WEAK_CALL_NAMES}}`), e/o
- chiamate con un argomento-costante debole (`{{BAD_CONSTANTS}}` , match case-insensitive) — es. `hashlib.new("md5")` .
- Usa la classe `StringLiteral` (NON `StrConst` , deprecata).

3.3 Sicurezza dell'iniezione

Il server valida ogni nome prima di iniettarlo:

- nomi funzione/metodo: regex `^[A-Za-z_][A-Za-z0-9_]*$` (`_render_names`)
- costanti: regex `^[A-Za-z0-9._-]+$` , minuscolizzate (`_render_consts`)

Nomi non validi vengono scartati → impossibile rompere o iniettare QL arbitrario. Ogni query renderizzata viene scritta in `generated_queries/` (pack valido con `qlpack.yml` → `codeql/python-all`) ed eseguita.

4. Knowledge base CWE a due layer (`knowledge/cwe_wiki.json`)

Ogni CWE ha **due layer distinti e complementari**:

Layer	Campi	Fonte	Mantenuto da
Ufficiale ("cos'è")	<code>mitre_description</code> , <code>mitre_extended_description</code> , <code>mitre_mitigations</code> , <code>references</code>	MITRE REST API	<code>fetch_mitre.py</code>
Azioni ("cosa fa l'agente")	<code>actions</code> : tool, <code>candidate_*_names</code> , <code>how_to_verify</code> , <code>interpretation</code> , <code>false_positive_notes</code>	curato da noi	<code>build_actions.py</code>

Più i campi curati di base: `detection` (`taint` | `api_misuse`), `description` , `taint_intuition` / `detection_intuition` , i nomi candidati, `remediation` .

Il tool `cwe_knowledge(cwe)` restituisce un payload strutturato in 3 sezioni: `official` + `our_knowledge` + `actions` .

5. Downloader MITRE (`fetch_mitre.py`)

Arricchimento one-shot dalla **REST API ufficiale** (nessuna credenziale):

- Endpoint: `https://cwe-api.mitre.org/api/v1/cwe/weakness/{ids}` — risposta JSON.
- Aggiunge solo i campi `mitre_*` **senza toccare** i nomi curati.
- Dimensione: pochi KB per CWE (l'XML completo $\approx$ 2 MB se servisse tutto il DB).

```
python fetch_mitre.py                # arricchisce tutti i CWE
nel wiki
python fetch_mitre.py CWE-89 CWE-327
```

6. Generatore delle azioni (`build_actions.py`)

Mantiene la nostra KB sulle azioni. **Idempotente**: riempie solo le voci senza `actions`, preservando le rifiniture manuali.

```
python build_actions.py                # riempie i mancanti
python build_actions.py --force        # rigenera tutti
```

Le note false-positive si curano nel dict `FP_NOTES` (es. CWE-330: "random spesso non-security, conferma che il valore sia sensibile").

7. Report ancorato (anti-allucinazione CWE)

Il CWE nel report **non è asserito dal modello** ma deriva deterministicamente dal SARIF:

1. `run_taint_query(cwe=...)` / `run_api_misuse_query(cwe=...)` incidono `@tags` `external/cwe/cwe-NNN` nella query generata.
2. `_parse_sarif` $\rightarrow$ `_cwe_from_tags` riestrae il CWE **dal tag SARIF**.
3. `_extract_flow` estrae il path **source $\rightarrow$ sink** (codeFlows) con file:riga come evidenza (per le taint).

Il system prompt impone, per ogni finding: **Evidenza deterministica (path CodeQL) / Classificazione (CWE dal finding) / Remediation**. Vietato indovinare il CWE.

8. Tutti i tool MCP (14)

Tool	Scopo
<code>list_python_files</code>	enumera i .py della repo
<code>create_codeql_database</code>	costruisce il DB CodeQL
<code>analyze_database</code>	suite di sicurezza standard (copertura ampia)
<code>run_custom_query</code>	esegue un .ql arbitrario
<code>check_sql_injection</code> ... <code>check_xss</code>	5 query CWE curate, fisse
<code>run_taint_query</code>	template taint name-based (sink/source/sanitizer + cwe)
<code>run_api_misuse_query</code>	template crypto name-based (weak_call_names/bad_constants + cwe)
<code>list_cwes</code>	indice CWE con detection kind + tool giusto
<code>cwe_knowledge</code>	pagina wiki a due layer (official + our_knowledge + actions)
<code>read_file_snippet</code>	legge righe sorgente attorno a un finding

9. Come estendere a un nuovo CWE

1. Aggiungi la voce a `knowledge/cwe_wiki.json` (detection + nomi candidati)
2. `python fetch_mitre.py CWE-XXX` # layer ufficiale
3. `python build_actions.py` # layer **azioni** (solo mancanti)

Per una nuova famiglia che non sia taint né crypto, serve un nuovo template + tool (come fatto per api_misuse).